ชุดโมเดล all_197

197 โมเดล · pass 118 / fail 79 · 11 หัวข้อ · ยังไม่ได้รัน Rodin

1. ภาพรวม

รวมทุกโมเดลไว้ที่เดียว: **รอบแรก 47 ตัว** (model_002–048, ใช้ ID เดิม; แก้ใหม่ 21 ตัว ดูข้อ 6) + **ชุดใหม่ 150 ตัว** (batch_02–04) — model_001 (Payment) เก็บไว้ทดสอบ ไม่ได้อยู่ในชุดนี้

แต่ละตัวเริ่มจาก sequence diagram แล้วแปลงเป็น Event-B ด้วย ChronoSeq-B **โดยไม่แก้ Event-B ภายหลัง** (เอา diagram ไปแปลงใหม่จะได้ไฟล์เดิมทุกตัวอักษร)

- **pass** — diagram ถูก
- **fail** — diagram มีจุดผิดที่ตั้งใจใส่ไว้ ทำให้ Event-B ที่แปลงออกมามี PO ที่พิสูจน์ไม่ได้

> **pass / fail ในชุดนี้คือผลที่ออกแบบไว้ ยังไม่ใช่ผลจาก Rodin**
> ผลจริงได้หลังรัน Rodin (ML/PP) และจะอยู่ใน `by_result/`

2. โครงสร้าง folder

```
batches/all_197/
├── README_all_197.pdf            ← ไฟล์นี้
├── Rodin_Import_all_197.zip      ← import เข้า Rodin ครั้งเดียว ครบ 197 project
├── 01_การเงินและการชำระเงิน/       ← หัวข้อ
│   ├── pass/
│   │   └── <ชื่อโมเดล>/
│   │       ├── <ชื่อโมเดล>.drawio.xml
│   │       ├── <ชื่อโมเดล>_EventB.txt
│   │       ├── <ชื่อโมเดล>_Rodin.zip
│   │       └── meta.json
│   └── fail/
├── 02_สุขภาพและการแพทย์/
└── ... (รวม 11 หัวข้อ)
```

ไฟล์	คืออะไร
`.drawio.xml`	sequence diagram (ของ fail มีจุดผิดอยู่ในนี้)
`_EventB.txt`	Event-B ที่ได้จากสูตรแปลง = input ของ LLM
`_Rodin.zip`	Rodin project ของโมเดลนี้
`meta.json`	pass/fail ที่ออกแบบ, จุดผิด, PO ที่คาดว่าจะล้ม — **ห้ามใช้เป็น input ของ LLM** เพราะมีเฉลย
`Rodin_Import_all_197.zip`	ทุก project ในไฟล์เดียว (Rodin: File → Import → Existing Projects into Workspace → Select archive file)

ขั้นตอนต่อไป

1. import `Rodin_Import_all_197.zip` ในเครื่องที่มี Rodin + Atelier B (ML/PP)
2. รอ auto-prover จนครบ แล้ว zip workspace ส่งกลับ
3. รัน `python tools/generators/integrate_batch.py --batch all_197 --workspace <workspace>` → รอบแรกแทนที่ ID เดิม ชุดใหม่ได้ model_049–model_198 พร้อม label และ by_result/

3. จำนวนตามหัวข้อ

หัวข้อ	pass	fail	รวม
01_การเงินและการชำระเงิน	9	4	13
02_สุขภาพและการแพทย์	12	14	26
03_การเดินทางและยานพาหนะ	18	8	26
04_โลจิสติกส์และคลังสินค้า	9	5	14
05_บ้านและอาคารอัจฉริยะ	11	10	21
06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน	8	5	13
07_ค้าปลีก อาหาร และ e-commerce	9	5	14
08_การจอง ที่พัก และกิจกรรม	13	6	19
09_การศึกษาและบริการภาครัฐ	10	7	17
10_ไอทีและบริการดิจิทัล	10	9	19
11_เกษตรและสิ่งแวดล้อม	9	6	15
รวม	**118**	**79**	**197**

4. อ่านชื่อ PO

รูปแบบ `<event>/<label>/<ชนิด>`

ชนิด	ต้องพิสูจน์ว่า	ตัวอย่าง
`INV`	หลัง event ทำงาน invariant ยังจริง	`receiveX_3/inv5/INV`
`FIS`	action `:∈` มีค่าให้เลือกอย่างน้อย 1 ค่า	`INITIALISATION/act10/FIS`
`WD`	สูตรใน guard คำนวณได้ เช่น `a mod b` ต้องมี a ≥ 0 และ b > 0	`sendX_4/grd5/WD`

5. diagram ผิดแบบไหนทำให้ PO ล้ม

สูตรแปลงสร้าง invariant แค่ 2 แบบ คือ ชนิดข้อมูล (`x ∈ ℤ`, ⊆) และ `inv5: receivedMessages ⊆ sentMessages` ซึ่งสูตรใส่ guard รองรับให้เองเสมอ **INV จึงไม่ล้มไม่ว่าวาด diagram แบบไหน** จุดใน diagram ที่ทำให้ PO ล้มได้มี 2 แบบ:

กลุ่ม	เขียนใน diagram	สูตรแปลงได้	PO ที่ล้ม
A. ช่วงเวลากลับด้าน	`notify() {5..3}`	`t_time_i :∈ 5..3`	**FIS** — ไม่มีค่าให้เลือก (โมเดลผิดจริง)
B. mod กับตัวนับ loop	loop `[retryNo < 5]` + alt/opt `[retryNo mod 2 == 0]`	guard `retryNo mod 2 = 0` + invariant แค่ `retryNo ∈ ℤ`	**WD** — ต้องรู้ว่า retryNo ≥ 0 (ค่าจริงถูก แต่พิสูจน์ไม่ได้)

ชนิด	กลุ่ม	จุดผิดใน diagram	ผล	จำนวน
`inverted1`	A	ช่วงเวลา {a..b} ของ message 1 ตัวเขียนกลับด้าน (a > b)	สูตรแปลงได้ t_time :∈ a..b ซึ่งไม่มีค่าให้เลือก → FIS ล้ม 1 ข้อ	18
`inverted2`	A	เหมือน inverted1 แต่กลับด้าน 2 message	FIS ล้ม 2 ข้อ	15
`inverted3`	A	เหมือน inverted1 แต่กลับด้าน 3 message	FIS ล้ม 3 ข้อ	7

`inverted_frame`	A	ช่วงเวลากลับด้านบน message ที่อยู่ใน fragment (loop / alt / opt / par)	FIS ล้ม 1 ข้อ — fragment ไม่ได้เปลี่ยนผล	14
`mod_alt`	B	alt ใช้เงื่อนไข [c mod k == 0] / [c mod k != 0] กับตัวนับ loop c	c mod k ต้องมี c ≥ 0 แต่สูตรแปลงให้แค่ c ∈ ℤ → WD ล้มทุก guard ที่มี mod	9
`mod_opt`	B	opt ใช้เงื่อนไข [c mod k == 0] กับตัวนับ loop c	เหตุผลเดียวกับ mod_alt	10
`mod_inverted`	A+B	มีทั้งช่วงเวลากลับด้าน 1 จุด และเงื่อนไข mod กับตัวนับ loop	FIS ล้ม 1 ข้อ + WD ล้ม	6

6. รอบแรก: ตัวที่แก้ใหม่

รอบแรกมี 21 ตัวที่ Event-B ไม่ได้มาจาก diagram ตรง ๆ (แก้ Event-B หลังแปลง หรือเติม invariant เอง) ตอนนี้แก้ให้ทุกตัวเป็นผลของสูตรแปลงแล้ว ของเดิมเก็บไว้ที่ `archive/round1_post_conversion_edits/`

ID	โมเดล	เดิม	ตอนนี้
model_002	AirportBaggage	`mutated: t_time_2 ≥ 0, t_delay_2 ≥ 0, t_success_2 ≥ 0 added; t_delay_2 initialised to −2`	fail: `inverted_frame` (จุดผิดใน diagram)
model_004	BankTransfer	`mutated: removed enabling guard ` m ∈ sentMessages` from two receive events`	fail: `inverted2` (จุดผิดใน diagram)
model_005	BatteryChargeCycle	`mutated: pulseNo ≥ 0 and pulseNo ≤ 8 added; checkloop decrements the counter instead of incrementing`	fail: `inverted_frame` (จุดผิดใน diagram)
model_006	BloodBankRequest	`augmented: added currentMessage ⊆ sentMessages; t_time_4 ≥ 0; t_delay_4 ≥ 0`	pass: แปลงจาก diagram ตรง ๆ
model_008	ChatMessaging	`mutated: added receiverdataMessages ⊆ senderdataMessages (true when reachable, not inductive)`	fail: `inverted3` (จุดผิดใน diagram)
model_011	ColdChainMonitor	`mutated: t_time_2 ≥ 0 (provable) and t_success_2 ≥ 0 (true, but needs a missing t_delay_2 ≥ 0)`	fail: `inverted1` (จุดผิดใน diagram)
model_012	ConferenceRoomBooking	`mutated: dom(receiverdataMessages) ⊆ receivedMessages added, but every send resets receivedMessages to ∅`	fail: `inverted_frame` (จุดผิดใน diagram)
model_013	CourseEnrollment	`mutated: currentMessage ⊆ sentMessages (provable) and receivedMessages ⊆ dom(receiverdataMessages) (wrong: assumes every message carries data)`	fail: `inverted2` (จุดผิดใน diagram)
model_020	FitnessTracker	`augmented: added syncRound ≤ 10; t_time_3 ≥ 0; t_delay_3 ≥ 0; t_success_3 ≥ 0`	pass: แปลงจาก diagram ตรง ๆ
model_022	GymCheckIn	`mutated: INITIALISATION marks scanQR_1 as received while nothing has been sent`	fail: `inverted1` (จุดผิดใน diagram)
model_024	HotelCheckIn	`augmented: added currentMessage ⊆ sentMessages; receivedMessages ⊆ dom(receiver)`	pass: แปลงจาก diagram ตรง ๆ

model_028	LoginRateLimiter	`mutated: off-by-one: checkloop guard failCount < 5 weakened to failCount ≤ 5 against failCount ≤ 5`	fail: `mod_opt` (จุดผิดใน diagram)
model_029	MedicationDispenser	`mutated: currentMessage ⊆ sentMessages added; send of logDose_4 sets currentMessage to {logAck_5}`	fail: `inverted1` (จุดผิดใน diagram)
model_030	OnlineAuction	`mutated: removed enabling guard` `m ∈ sentMessages` `from receive of saleConfirm_6`	fail: `inverted_frame` (จุดผิดใน diagram)
model_034	PrinterQueue	`mutated: loop-counter bounds added; retryPrint initialised to 4 against retryPrint ≤ 3`	fail: `inverted_frame` (จุดผิดใน diagram)
model_036	ServerAutoScaling	`augmented: added checkNo ≤ 12; currentMessage ⊆ sentMessages`	pass: แปลงจาก diagram ตรง ๆ
model_039	SmartMeterBilling	`mutated: copy-paste slip: receive of generateBill_3 records payBill_4 (not yet sent) as received`	fail: `inverted2` (จุดผิดใน diagram)
model_041	SmartPlugControl	`mutated: receive of turnOff_3 records turnOn_5 (other alt branch, never sent) as received`	fail: `inverted1` (จุดผิดใน diagram)
model_042	TaxiDispatch	`mutated: send of findDriver_2 sets receivedMessages to {tripComplete_6} (not yet sent) instead of ∅`	fail: `inverted2` (จุดผิดใน diagram)
model_044	ToolRentalLocker	`mutated: INITIALISATION marks codeValid_5 received; receive of verifyCode_4 records releaseTool_6`	fail: `inverted1` (จุดผิดใน diagram)
model_045	TrafficLightController	`mutated: timing invariant tighter than the model allows (t_success_1 ≤ 20, reachable max 30+5)`	fail: `inverted2` (จุดผิดใน diagram)

7. จุดผิดของโมเดล fail ทีละตัว

ใน diagram = ข้อความที่ผิดใน .drawio.xml · **ใน Event-B** = บรรทัดที่ได้จากจุดนั้นใน _EventB.txt

01_การเงินและการชำระเงิน (4)

BankTransfer model_004 — โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted2`

`01_การเงินและการชำระเงิน/fail/BankTransfer/`

ใน diagram	ช่วงเวลากลับด้าน: `checkFraud(txnRef) {5..1}`, `postTransfer(txnRef) {4..2}`
ใน Event-B	บรรทัด 49 `act12: t_time_2 :∈ 5..1` บรรทัด 52 `act15: t_time_4 :∈ 4..2`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 5..1` `INITIALISATION/act15/FIS` — `act15: t_time_4 :∈ 4..2`
ทำไม	สูตรแปลงได้ `:∈ 5..1` ซึ่งไม่มีค่าให้เลือก (5 > 1); สูตรแปลงได้ `:∈ 4..2` ซึ่งไม่มีค่าให้เลือก (4 > 2) → FIS ล้ม

FleetFuelCard

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted_frame`

`01_การเงินและการชำระเงิน/fail/FleetFuelCard/`

ใน diagram	ช่วงเวลากลับด้าน: `sendFuel(cardValue) {10..0}`
ใน Event-B	บรรทัด 57 `act13: t_time_3 :∈ 10..0`
PO ที่ล้ม (1)	`INITIALISATION/act13/FIS` — `act13: t_time_3 :∈ 10..0`
ทำไม	สูตรแปลงได้ `:∈ 10..0` ซึ่งไม่มีค่าให้เลือก (10 > 0) → FIS ล้ม

MobileTopUp

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted3`

`01_การเงินและการชำระเงิน/fail/MobileTopUp/`

ใน diagram	ช่วงเวลากลับด้าน: `notifyCredit() {10..3}`, `confirmBundle() {30..24}`, `ackCredit() {10..8}`
ใน Event-B	บรรทัด 49 `act9: t_time_1 :∈ 10..3` บรรทัด 52 `act12: t_time_2 :∈ 30..24` บรรทัด 55 `act15: t_time_4 :∈ 10..8`

PO ที่ล้ม (3)	`INITIALISATION/act9/FIS` — `act9: t_time_1 :∈ 10..3`
	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 30..24`
	`INITIALISATION/act15/FIS` — `act15: t_time_4 :∈ 10..8`

ทำไม	สูตรแปลงได้ `:∈ 10..3` ซึ่งไม่มีค่าให้เลือก (10 > 3); สูตรแปลงได้ `:∈ 30..24` ซึ่งไม่มีค่าให้เลือก (30 > 24); สูตรแปลงได้ `:∈ 10..8` ซึ่งไม่มีค่าให้เลือก (10 > 8) → FIS ล้ม

OnlineBankingOTP

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · `mod_alt`

`01_การเงินและการชำระเงิน/fail/OnlineBankingOTP/`

ใน diagram	alt ใช้เงื่อนไข `[retryNo mod 4 == 0]` / `[retryNo mod 4 != 0]` กับตัวนับของ loop `[retryNo < 12]`
ใน Event-B	บรรทัด 183 `grd5: retryNo mod 4 = 0` บรรทัด 201 `grd7: retryNo mod 4 = 0` บรรทัด 214 `grd5: retryNo mod 4 = 0` … และอีก 5 บรรทัด

PO ที่ล้ม (8)	`sendvalidateAccount_4/grd5/WD` — `grd5: retryNo mod 4 = 0`
	`receivevalidateAccount_4/grd7/WD` — `grd7: retryNo mod 4 = 0`
	`sendupdateOtp_5/grd5/WD` — `grd5: retryNo mod 4 = 0`
	… และอีก 5 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `retryNo mod 4` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

02_สุขภาพและการแพทย์ (14)

BloodDonationDrive

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

02_สุขภาพและการแพทย์/fail/BloodDonationDrive/

ใน diagram	ช่วงเวลากลับด้าน: `ackScreening(donationInfo) {5..4}`
ใน Event-B	บรรทัด 48 `act10: t_time_1 :∈ 5..4`
PO ที่ล้ม (1)	`INITIALISATION/act10/FIS` — `act10: t_time_1 :∈ 5..4`
ทำไม	สูตรแปลงได้ `:∈ 5..4` ซึ่งไม่มีค่าให้เลือก (5 > 4) → FIS ล้ม

DentalAppointment

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

02_สุขภาพและการแพทย์/fail/DentalAppointment/

ใน diagram	ช่วงเวลากลับด้าน: `confirmAppointment() {5..3}`
ใน Event-B	บรรทัด 59 `act17: t_time_5 :∈ 5..3`
PO ที่ล้ม (1)	`INITIALISATION/act17/FIS` — `act17: t_time_5 :∈ 5..3`
ทำไม	สูตรแปลงได้ `:∈ 5..3` ซึ่งไม่มีค่าให้เลือก (5 > 3) → FIS ล้ม

HospitalLabRequest

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

02_สุขภาพและการแพทย์/fail/HospitalLabRequest/

ใน diagram	alt ใช้เงื่อนไข `[cycleNo mod 3 == 0] / [cycleNo mod 3 != 0]` กับตัวนับของ loop `[cycleNo < 8]`
ใน Event-B	บรรทัด 158 `grd5: cycleNo mod 3 = 0` บรรทัด 176 `grd7: cycleNo mod 3 = 0` บรรทัด 189 `grd5: cycleNo mod 3 = 0` ... และอีก 10 บรรทัด

PO ที่ล้ม (13)	`sendscheduleOrder_4/grd5/WD` — `grd5: cycleNo mod 3 = 0`
	`receivescheduleOrder_4/grd7/WD` — `grd7: cycleNo mod 3 = 0`
	`sendackSample_5/grd5/WD` — `grd5: cycleNo mod 3 = 0`
	... และอีก 10 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `cycleNo mod 3` คำนวณได้เมื่อ `cycleNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `cycleNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

LabSampleTesting model_026

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

02_สุขภาพและการแพทย์/fail/LabSampleTesting/

ใน diagram	ช่วงเวลากลับด้าน: `transportSample(sampleID) {12..4}`, `finalReport(reportID) {9..6}`
ใน Event-B	บรรทัด 52 `act12: t_time_2 :∈ 12..4` บรรทัด 58 `act18: t_time_6 :∈ 9..6`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 12..4` `INITIALISATION/act18/FIS` — `act18: t_time_6 :∈ 9..6`
ทำไม	สูตรแปลงได้ `:∈ 12..4` ซึ่งไม่มีค่าให้เลือก (12 > 4); สูตรแปลงได้ `:∈ 9..6` ซึ่งไม่มีค่าให้เลือก (9 > 6) → FIS ล้ม

MedicalImagingPACS

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

02_สุขภาพและการแพทย์/fail/MedicalImagingPACS/

ใน diagram	alt ใช้เงื่อนไข `[roundNo mod 3 == 0]` / `[roundNo mod 3 != 0]` กับตัวนับของ loop `[roundNo < 6]`
ใน Event-B	บรรทัด 141 `grd5: roundNo mod 3 = 0` บรรทัด 159 `grd7: roundNo mod 3 = 0` บรรทัด 173 `grd5: roundNo mod 3 ≠ 0` บรรทัด 191 `grd7: roundNo mod 3 ≠ 0`

PO ที่ล้ม (4)	`sendapproveReport_4/grd5/WD` — `grd5: roundNo mod 3 = 0`
	`receiveapproveReport_4/grd7/WD` — `grd7: roundNo mod 3 = 0`
	`sendrequestImage_5/grd5/WD` — `grd5: roundNo mod 3 ≠ 0`
	`receiverequestImage_5/grd7/WD` — `grd7: roundNo mod 3 ≠ 0`

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `roundNo mod 3` คำนวณได้เมื่อ `roundNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `roundNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

MedicationDispenser model_029

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

02_สุขภาพและการแพทย์/fail/MedicationDispenser/

ใน diagram	ช่วงเวลากลับด้าน: `doseAlert(pillCount) {60..35}`
ใน Event-B	บรรทัด 49 `act12: t_time_2 :∈ 60..35`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 60..35`
ทำไม	สูตรแปลงได้ `:∈ 60..35` ซึ่งไม่มีค่าให้เลือก (60 > 35) → FIS ล้ม

PatientIntakeForm

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

02_สุขภาพและการแพทย์/fail/PatientIntakeForm/

ใน diagram	opt ใช้เงื่อนไข `[attemptNo mod 2 == 0]` กับตัวนับของ loop `[attemptNo < 5]`
ใน Event-B	บรรทัด 149 `grd5: attemptNo mod 2 = 0` บรรทัด 167 `grd7: attemptNo mod 2 = 0`
PO ที่ล้ม (2)	`sendreportForm_4_opt1/grd5/WD` — `grd5: attemptNo mod 2 = 0` `receivereportForm_4_opt1/grd7/WD` — `grd7: attemptNo mod 2 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `attemptNo mod 2` คำนวณได้เมื่อ `attemptNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `attemptNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

PharmacyDelivery

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

02_สุขภาพและการแพทย์/fail/PharmacyDelivery/

ใน diagram	ช่วงเวลากลับด้าน: `scheduleSignature(packageCode) {3..0}`, `approveRoute(packageRef) {3..2}`
ใน Event-B	บรรทัด 67 `act19: t_time_11 :∈ 3..0` บรรทัด 70 `act22: t_time_12 :∈ 3..2`
PO ที่ล้ม (2)	`INITIALISATION/act19/FIS` — `act19: t_time_11 :∈ 3..0` `INITIALISATION/act22/FIS` — `act22: t_time_12 :∈ 3..2`
ทำไม	สูตรแปลงได้ `:∈ 3..0` ซึ่งไม่มีค่าให้เลือก (3 > 0); สูตรแปลงได้ `:∈ 3..2` ซึ่งไม่มีค่าให้เลือก (3 > 2) → FIS ล้ม

PharmacyRefill

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

02_สุขภาพและการแพทย์/fail/PharmacyRefill/

ใน diagram	ช่วงเวลากลับด้าน: `checkClaim(claimCode) {2..1}`, `queryRefill(prescriptionCode) {30..8}`, `ackRefill(claimData) {30..5}`
ใน Event-B	บรรทัด 57 `act10: t_time_2 :∈ 2..1` บรรทัด 60 `act13: t_time_4 :∈ 30..8` บรรทัด 63 `act16: t_time_5 :∈ 30..5`

PO ที่ล้ม (3)	`INITIALISATION/act10/FIS` — `act10: t_time_2 :∈ 2..1`
	`INITIALISATION/act13/FIS` — `act13: t_time_4 :∈ 30..8`
	`INITIALISATION/act16/FIS` — `act16: t_time_5 :∈ 30..5`

ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1); สูตรแปลงได้ `:∈ 30..8` ซึ่งไม่มีค่าให้เลือก (30 > 8); สูตรแปลงได้ `:∈ 30..5` ซึ่งไม่มีค่าให้เลือก (30 > 5) → FIS ล้ม

RemotePatientMonitor

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

02_สุขภาพและการแพทย์/fail/RemotePatientMonitor/

ใน diagram	ช่วงเวลากลับด้าน: `notifyAlert(alertCode) {60..52}`
ใน Event-B	บรรทัด 63 `act19: t_time_8 :∈ 60..52`
PO ที่ล้ม (1)	`INITIALISATION/act19/FIS` — `act19: t_time_8 :∈ 60..52`
ทำไม	สูตรแปลงได้ `:∈ 60..52` ซึ่งไม่มีค่าให้เลือก (60 > 52) → FIS ล้ม

RuralTelehealthKiosk

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · `mod_alt`

`02_สุขภาพและการแพทย์/fail/RuralTelehealthKiosk/`

ใน diagram	alt ใช้เงื่อนไข `[passNo mod 4 == 0] / [passNo mod 4 != 0]` กับตัวนับของ loop `[passNo < 6]`
ใน Event-B	บรรทัด 172 `grd5: passNo mod 4 = 0` บรรทัด 189 `grd7: passNo mod 4 = 0` บรรทัด 202 `grd5: passNo mod 4 = 0` ... และอีก 13 บรรทัด

PO ที่ล้ม (16)	`sendschedulePrescription_4/grd5/WD` — `grd5: passNo mod 4 = 0`
	`receiveschedulePrescription_4/grd7/WD` — `grd7: passNo mod 4 = 0`
	`sendcancelPrescription_5/grd5/WD` — `grd5: passNo mod 4 = 0`
	... และอีก 13 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `passNo mod 4` คำนวณได้เมื่อ `passNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `passNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

SmartMirrorFitness

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · `mod_opt`

`02_สุขภาพและการแพทย์/fail/SmartMirrorFitness/`

ใน diagram	opt ใช้เงื่อนไข `[attemptNo mod 2 == 0]` กับตัวนับของ loop `[attemptNo < 6]`
ใน Event-B	บรรทัด 172 `grd5: attemptNo mod 2 = 0` บรรทัด 189 `grd7: attemptNo mod 2 = 0` บรรทัด 202 `grd5: attemptNo mod 2 = 0` ... และอีก 11 บรรทัด

PO ที่ล้ม (14)	`sendconfirmWorkout_4_opt1/grd5/WD` — `grd5: attemptNo mod 2 = 0`
	`receiveconfirmWorkout_4_opt1/grd7/WD` — `grd7: attemptNo mod 2 = 0`
	`sendlogWorkout_5_opt1/grd5/WD` — `grd5: attemptNo mod 2 = 0`
	... และอีก 11 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `attemptNo mod 2` คำนวณได้เมื่อ `attemptNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `attemptNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

TelemedicineVisit

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

02_สุขภาพและการแพทย์/fail/TelemedicineVisit/

ใน diagram	alt ใช้เงื่อนไข `[stepNo mod 2 == 0]` / `[stepNo mod 2 != 0]` กับตัวนับของ loop `[stepNo < 2]`
ใน Event-B	บรรทัด 138 `grd5: stepNo mod 2 = 0` บรรทัด 156 `grd7: stepNo mod 2 = 0` บรรทัด 169 `grd5: stepNo mod 2 ≠ 0` บรรทัด 187 `grd7: stepNo mod 2 ≠ 0`

PO ที่ล้ม (4)	`sendschedulePrescription_4/grd5/WD` — `grd5: stepNo mod 2 = 0`
	`receiveschedulePrescription_4/grd7/WD` — `grd7: stepNo mod 2 = 0`
	`sendrequestVisit_5/grd5/WD` — `grd5: stepNo mod 2 ≠ 0`
	`receiverequestVisit_5/grd7/WD` — `grd7: stepNo mod 2 ≠ 0`

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `stepNo mod 2` คำนวณได้เมื่อ `stepNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `stepNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

VaccineColdBox

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

02_สุขภาพและการแพทย์/fail/VaccineColdBox/

ใน diagram	ช่วงเวลากลับด้าน: `syncReading(alarmValue) {10..1}`, `approveAlarm(vaccineRef) {10..6}`, `sendVaccine() {30..13}`
ใน Event-B	บรรทัด 53 `act11: t_time_3 :∈ 10..1` บรรทัด 59 `act17: t_time_5 :∈ 10..6` บรรทัด 62 `act20: t_time_6 :∈ 30..13`

PO ที่ล้ม (3)	`INITIALISATION/act11/FIS` — `act11: t_time_3 :∈ 10..1`
	`INITIALISATION/act17/FIS` — `act17: t_time_5 :∈ 10..6`
	`INITIALISATION/act20/FIS` — `act20: t_time_6 :∈ 30..13`

ทำไม	สูตรแปลงได้ `:∈ 10..1` ซึ่งไม่มีค่าให้เลือก (10 > 1); สูตรแปลงได้ `:∈ 10..6` ซึ่งไม่มีค่าให้เลือก (10 > 6); สูตรแปลงได้ `:∈ 30..13` ซึ่งไม่มีค่าให้เลือก (30 > 13) → FIS ล้ม

03_การเดินทางและยานพาหนะ (8)

AirportBaggage model_002

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

03_การเดินทางและยานพาหนะ/fail/AirportBaggage/

ใน diagram	ช่วงเวลากลับด้าน: `loadBag(bagTag) {15..13}`
ใน Event-B	บรรทัด 52 `act15: t_time_4 :∈ 15..13`
PO ที่ล้ม (1)	`INITIALISATION/act15/FIS` — `act15: t_time_4 :∈ 15..13`
ทำไม	สูตรแปลงได้ `:∈ 15..13` ซึ่งไม่มีค่าให้เลือก (15 > 13) → FIS ล้ม

AirportSecurityLane

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

`03_การเดินทางและยานพาหนะ/fail/AirportSecurityLane/`

ใน diagram	ช่วงเวลากลับด้าน: `validateTray() {2..1}`, `updateTray(alertData, alertValue) {3..0}`
ใน Event-B	บรรทัด 49 `act12: t_time_2 :∈ 2..1` บรรทัด 52 `act15: t_time_5 :∈ 3..0`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 2..1` `INITIALISATION/act15/FIS` — `act15: t_time_5 :∈ 3..0`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1); สูตรแปลงได้ `:∈ 3..0` ซึ่งไม่มีค่าให้เลือก (3 > 0) → FIS ล้ม

BusTicketBooking

โมเดลผิดจริง · A + B · mod_inverted

`03_การเดินทางและยานพาหนะ/fail/BusTicketBooking/`

ใน diagram	ช่วงเวลากลับด้าน: `reserveSeat() {10..0}` opt ใช้เงื่อนไข `[retryNo mod 3 == 0]` กับตัวนับของ loop `[retryNo < 3]`
ใน Event-B	บรรทัด 46 `act9: t_time_4 :∈ 10..0` บรรทัด 142 `grd5: retryNo mod 3 = 0` บรรทัด 159 `grd7: retryNo mod 3 = 0` ... และอีก 2 บรรทัด

PO ที่ล้ม (5)	`INITIALISATION/act9/FIS` — `act9: t_time_4 :∈ 10..0`
	`sendreserveSeat_4_opt1/grd5/WD` — `grd5: retryNo mod 3 = 0`
	`receivereserveSeat_4_opt1/grd7/WD` — `grd7: retryNo mod 3 = 0`
	... และอีก 2 ข้อ

ทำไม	สูตรแปลงได้ `:∈ 10..0` ซึ่งไม่มีค่าให้เลือก (10 > 0) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `retryNo mod 3` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

FlightCrewScheduling

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

`03_การเดินทางและยานพาหนะ/fail/FlightCrewScheduling/`

ใน diagram	ช่วงเวลากลับด้าน: `ackDuty(swapCode) {10..5}`, `assignSwap(dutyData) {8..4}`
ใน Event-B	บรรทัด 50 `act11: t_time_2 :∈ 10..5` บรรทัด 53 `act14: t_time_3 :∈ 8..4`
PO ที่ล้ม (2)	`INITIALISATION/act11/FIS` — `act11: t_time_2 :∈ 10..5` `INITIALISATION/act14/FIS` — `act14: t_time_3 :∈ 8..4`
ทำไม	สูตรแปลงได้ `:∈ 10..5` ซึ่งไม่มีค่าให้เลือก (10 > 5); สูตรแปลงได้ `:∈ 8..4` ซึ่งไม่มีค่าให้เลือก (8 > 4) → FIS ล้ม

RideHailingSurge

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

`03_การเดินทางและยานพาหนะ/fail/RideHailingSurge/`

ใน diagram	ช่วงเวลากลับด้าน: `requestTrip() {2..1}`
ใน Event-B	บรรทัด 44 `act11: t_time_4 :∈ 2..1`
PO ที่ล้ม (1)	`INITIALISATION/act11/FIS` — `act11: t_time_4 :∈ 2..1`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม

TaxiDispatch model_042

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

`03_การเดินทางและยานพาหนะ/fail/TaxiDispatch/`

ใน diagram	ช่วงเวลากลับด้าน: `findDriver(pickupLoc) {30..20}`, `tripComplete(fare) {60..7}`
ใน Event-B	บรรทัด 52 `act12: t_time_2 :∈ 30..20` บรรทัด 55 `act15: t_time_6 :∈ 60..7`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 30..20` `INITIALISATION/act15/FIS` — `act15: t_time_6 :∈ 60..7`
ทำไม	สูตรแปลงได้ `:∈ 30..20` ซึ่งไม่มีค่าให้เลือก (30 > 20); สูตรแปลงได้ `:∈ 60..7` ซึ่งไม่มีค่าให้เลือก (60 > 7) → FIS ล้ม

TrafficLightController model_045

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

`03_การเดินทางและยานพาหนะ/fail/TrafficLightController/`

ใน diagram	ช่วงเวลากลับด้าน: `setGreen(greenTime) {30..8}`, `setRed() {60..26}`
ใน Event-B	บรรทัด 46 `act9: t_time_1 :∈ 30..8` บรรทัด 52 `act15: t_time_4 :∈ 60..26`
PO ที่ล้ม (2)	`INITIALISATION/act9/FIS` — `act9: t_time_1 :∈ 30..8` `INITIALISATION/act15/FIS` — `act15: t_time_4 :∈ 60..26`
ทำไม	สูตรแปลงได้ `:∈ 30..8` ซึ่งไม่มีค่าให้เลือก (30 > 8); สูตรแปลงได้ `:∈ 60..26` ซึ่งไม่มีค่าให้เลือก (60 > 26) → FIS ล้ม

TrainSeatReservation

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

`03_การเดินทางและยานพาหนะ/fail/TrainSeatReservation/`

ใน diagram	alt ใช้เงื่อนไข `[passNo mod 2 == 0] / [passNo mod 2 != 0]` กับตัวนับของ loop `[passNo < 11]`
ใน Event-B	บรรทัด 163 `grd5: passNo mod 2 = 0` บรรทัด 181 `grd7: passNo mod 2 = 0` บรรทัด 194 `grd5: passNo mod 2 = 0` … และอีก 7 บรรทัด

PO ที่ล้ม (10)	`sendconfirmSeat_4/grd5/WD` — `grd5: passNo mod 2 = 0`
	`receiveconfirmSeat_4/grd7/WD` — `grd7: passNo mod 2 = 0`
	`sendreserveTicket_5/grd5/WD` — `grd5: passNo mod 2 = 0`
	… และอีก 7 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `passNo mod 2` คำนวณได้เมื่อ `passNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `passNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

04_โลจิสติกส์และคลังสินค้า (5)

CargoShipTracking

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

`04_โลจิสติกส์และคลังสินค้า/fail/CargoShipTracking/`

ใน diagram	ช่วงเวลากลับด้าน: `releaseContainer() {20..1}`
ใน Event-B	บรรทัด 60 `act19: t_time_6 :∈ 20..1`
PO ที่ล้ม (1)	`INITIALISATION/act19/FIS` — `act19: t_time_6 :∈ 20..1`
ทำไม	สูตรแปลงได้ `:∈ 20..1` ซึ่งไม่มีค่าให้เลือก (20 > 1) → FIS ล้ม

ColdChainMonitor model_011

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

04_โลจิสติกส์และคลังสินค้า/fail/ColdChainMonitor/

ใน diagram	ช่วงเวลากลับด้าน: `uploadLog(logBatch) {30..14}`
ใน Event-B	บรรทัด 49 `act12: t_time_2 :∈ 30..14`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 30..14`
ทำไม	สูตรแปลงได้ `:∈ 30..14` ซึ่งไม่มีค่าให้เลือก (30 > 14) → FIS ล้ม

InventoryReorder

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

04_โลจิสติกส์และคลังสินค้า/fail/InventoryReorder/

ใน diagram	opt ใช้เงื่อนไข `[retryNo mod 3 == 0]` กับตัวนับของ loop `[retryNo < 12]`
ใน Event-B	บรรทัด 151 `grd5: retryNo mod 3 = 0` บรรทัด 169 `grd7: retryNo mod 3 = 0` บรรทัด 182 `grd5: retryNo mod 3 = 0` บรรทัด 199 `grd7: retryNo mod 3 = 0`

PO ที่ล้ม (4)	`sendackInvoice_4_opt1/grd5/WD` — `grd5: retryNo mod 3 = 0`
	`receiveackInvoice_4_opt1/grd7/WD` — `grd7: retryNo mod 3 = 0`
	`sendlogStock_5_opt1/grd5/WD` — `grd5: retryNo mod 3 = 0`
	`receivelogStock_5_opt1/grd7/WD` — `grd7: retryNo mod 3 = 0`

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `retryNo mod 3` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

ParcelLocker model_033

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

04_โลจิสติกส์และคลังสินค้า/fail/ParcelLocker/

ใน diagram	ช่วงเวลากลับด้าน: `sendPickupCode(pickupCode) {10..3}`
ใน Event-B	บรรทัด 49 `act12: t_time_3 :∈ 10..3`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 10..3`
ทำไม	สูตรแปลงได้ `:∈ 10..3` ซึ่งไม่มีค่าให้เลือก (10 > 3) → FIS ล้ม

ParcelReturnDropoff

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

04_โลจิสติกส์และคลังสินค้า/fail/ParcelReturnDropoff/

ใน diagram	ช่วงเวลากลับด้าน: `logRefund(refundID) {3..1}`
ใน Event-B	บรรทัด 51 `act13: t_time_7 :∈ 3..1`
PO ที่ล้ม (1)	`INITIALISATION/act13/FIS` — `act13: t_time_7 :∈ 3..1`
ทำไม	สูตรแปลงได้ `:∈ 3..1` ซึ่งไม่มีค่าให้เลือก (3 > 1) → FIS ล้ม

05_บ้านและอาคารอัจฉริยะ (10)

HomeSecurityAlarm

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

`05_บ้านและอาคารอัจฉริยะ/fail/HomeSecurityAlarm/`

ใน diagram	ช่วงเวลากลับด้าน: `notifyZone(alertInfo) {20..14}`
ใน Event-B	บรรทัด 57 `act16: t_time_4 :∈ 20..14`
PO ที่ล้ม (1)	`INITIALISATION/act16/FIS` — `act16: t_time_4 :∈ 20..14`
ทำไม	สูตรแปลงได้ `:∈ 20..14` ซึ่งไม่มีค่าให้เลือก (20 > 14) → FIS ล้ม

LiftEmergencyCall

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

`05_บ้านและอาคารอัจฉริยะ/fail/LiftEmergencyCall/`

ใน diagram	ช่วงเวลากลับด้าน: `releaseCall(faultCode) {8..2}`, `notifyFault() {60..46}`, `sendFault(faultData) {3..2}`
ใน Event-B	บรรทัด 56 `act11: t_time_1 :∈ 8..2` บรรทัด 59 `act14: t_time_5 :∈ 60..46` บรรทัด 62 `act17: t_time_8 :∈ 3..2`

PO ที่ล้ม (3)	`INITIALISATION/act11/FIS` — `act11: t_time_1 :∈ 8..2`
	`INITIALISATION/act14/FIS` — `act14: t_time_5 :∈ 60..46`
	`INITIALISATION/act17/FIS` — `act17: t_time_8 :∈ 3..2`

ทำไม	สูตรแปลงได้ `:∈ 8..2` ซึ่งไม่มีค่าให้เลือก (8 > 2); สูตรแปลงได้ `:∈ 60..46` ซึ่งไม่มีค่าให้เลือก (60 > 46); สูตรแปลงได้ `:∈ 3..2` ซึ่งไม่มีค่าให้เลือก (3 > 2) → FIS ล้ม

LiftMaintenance

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

`05_บ้านและอาคารอัจฉริยะ/fail/LiftMaintenance/`

ใน diagram	ช่วงเวลากลับด้าน: `checkJob() {8..0}`, `cancelFault() {3..1}`
ใน Event-B	บรรทัด 43 `act9: t_time_4 :∈ 8..0` บรรทัด 46 `act12: t_time_6 :∈ 3..1`
PO ที่ล้ม (2)	`INITIALISATION/act9/FIS` — `act9: t_time_4 :∈ 8..0` `INITIALISATION/act12/FIS` — `act12: t_time_6 :∈ 3..1`
ทำไม	สูตรแปลงได้ `:∈ 8..0` ซึ่งไม่มีค่าให้เลือก (8 > 0); สูตรแปลงได้ `:∈ 3..1` ซึ่งไม่มีค่าให้เลือก (3 > 1) → FIS ล้ม

OfficeLightingControl

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

`05_บ้านและอาคารอัจฉริยะ/fail/OfficeLightingControl/`

ใน diagram	ช่วงเวลากลับด้าน: `syncSchedule() {30..11}`
ใน Event-B	บรรทัด 45 `act10: t_time_2 :∈ 30..11`
PO ที่ล้ม (1)	`INITIALISATION/act10/FIS` — `act10: t_time_2 :∈ 30..11`
ทำไม	สูตรแปลงได้ `:∈ 30..11` ซึ่งไม่มีค่าให้เลือก (30 > 11) → FIS ล้ม

RoboticLawnMower

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted_frame`

`05_บ้านและอาคารอัจฉริยะ/fail/RoboticLawnMower/`

ใน diagram	ช่วงเวลากลับด้าน: `cancelBlade(bladeID) {20..0}`
ใน Event-B	บรรทัด 52 `act12: t_time_3 :∈ 20..0`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 20..0`
ทำไม	สูตรแปลงได้ `:∈ 20..0` ซึ่งไม่มีค่าให้เลือก (20 > 0) → FIS ล้ม

SmartAquarium

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · `mod_alt`

`05_บ้านและอาคารอัจฉริยะ/fail/SmartAquarium/`

ใน diagram	alt ใช้เงื่อนไข `[cycleNo mod 2 == 0]` / `[cycleNo mod 2 != 0]` กับตัวนับของ loop `[cycleNo < 4]`
ใน Event-B	บรรทัด 157 `grd5: cycleNo mod 2 = 0` บรรทัด 175 `grd7: cycleNo mod 2 = 0` บรรทัด 188 `grd5: cycleNo mod 2 = 0` ... และอีก 3 บรรทัด

PO ที่ล้ม (6)	`sendapproveLight_4/grd5/WD` — `grd5: cycleNo mod 2 = 0`
	`receiveapproveLight_4/grd7/WD` — `grd7: cycleNo mod 2 = 0`
	`sendqueryFeed_5/grd5/WD` — `grd5: cycleNo mod 2 = 0`
	... และอีก 3 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `cycleNo mod 2` คำนวณได้เมื่อ `cycleNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `cycleNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

SmartCurtainControl

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted1`

`05_บ้านและอาคารอัจฉริยะ/fail/SmartCurtainControl/`

ใน diagram	ช่วงเวลากลับด้าน: `releaseLux() {2..0}`
ใน Event-B	บรรทัด 38 `act8: t_time_5 :∈ 2..0`
PO ที่ล้ม (1)	`INITIALISATION/act8/FIS` — `act8: t_time_5 :∈ 2..0`
ทำไม	สูตรแปลงได้ `:∈ 2..0` ซึ่งไม่มีค่าให้เลือก (2 > 0) → FIS ล้ม

SmartFridgeInventory

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · `inverted2`

`05_บ้านและอาคารอัจฉริยะ/fail/SmartFridgeInventory/`

ใน diagram	ช่วงเวลากลับด้าน: `queryItem(itemID) {15..2}`, `syncOrder() {8..3}`
ใน Event-B	บรรทัด 80 `act26: t_time_8 :∈ 15..2` บรรทัด 83 `act29: t_time_11 :∈ 8..3`
PO ที่ล้ม (2)	`INITIALISATION/act26/FIS` — `act26: t_time_8 :∈ 15..2` `INITIALISATION/act29/FIS` — `act29: t_time_11 :∈ 8..3`
ทำไม	สูตรแปลงได้ `:∈ 15..2` ซึ่งไม่มีค่าให้เลือก (15 > 2); สูตรแปลงได้ `:∈ 8..3` ซึ่งไม่มีค่าให้เลือก (8 > 3) → FIS ล้ม

SmartPlugControl model_041

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

05_บ้านและอาคารอัจฉริยะ/fail/SmartPlugControl/

ใน diagram	ช่วงเวลากลับด้าน: `plugState(powerStatus) {2..1}`
ใน Event-B	บรรทัด 46 `act9: t_time_2 :∈ 2..1`
PO ที่ล้ม (1)	`INITIALISATION/act9/FIS` — `act9: t_time_2 :∈ 2..1`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม

WaterLeakDetection

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

05_บ้านและอาคารอัจฉริยะ/fail/WaterLeakDetection/

ใน diagram	opt ใช้เงื่อนไข `[cycleNo mod 2 == 0]` กับตัวนับของ loop `[cycleNo < 10]`
ใน Event-B	บรรทัด 172 `grd5: cycleNo mod 2 = 0` บรรทัด 190 `grd7: cycleNo mod 2 = 0`
PO ที่ล้ม (2)	`sendreleaseLeak_4_opt1/grd5/WD` — `grd5: cycleNo mod 2 = 0` `receivereleaseLeak_4_opt1/grd7/WD` — `grd7: cycleNo mod 2 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `cycleNo mod 2` คำนวณได้เมื่อ `cycleNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `cycleNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน (5)

BatteryChargeCycle model_005

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน/fail/BatteryChargeCycle/

ใน diagram	ช่วงเวลากลับด้าน: `chargePulse(currentA) {2..1}`
ใน Event-B	บรรทัด 46 `act12: t_time_3 :∈ 2..1`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 2..1`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม

CraneLoadMonitor

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน/fail/CraneLoadMonitor/

ใน diagram	ช่วงเวลากลับด้าน: `cancelLimit(alarmRef) {2..1}`, `ackLimit(limitID) {5..4}`, `issueLimit() {10..7}`
ใน Event-B	บรรทัด 44 `act8: t_time_1 :∈ 2..1` บรรทัด 47 `act11: t_time_3 :∈ 5..4` บรรทัด 50 `act14: t_time_4 :∈ 10..7`

PO ที่ล้ม (3)	`INITIALISATION/act8/FIS` — `act8: t_time_1 :∈ 2..1`
	`INITIALISATION/act11/FIS` — `act11: t_time_3 :∈ 5..4`
	`INITIALISATION/act14/FIS` — `act14: t_time_4 :∈ 10..7`

ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1); สูตรแปลงได้ `:∈ 5..4` ซึ่งไม่มีค่าให้เลือก (5 > 4); สูตรแปลงได้ `:∈ 10..7` ซึ่งไม่มีค่าให้เลือก (10 > 7) → FIS ล้ม

SmartMeterBilling model_039

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน/fail/SmartMeterBilling/

ใน diagram	ช่วงเวลากลับด้าน: `sendUsage(kwhReading) {5..1}`, `generateBill(billAmount) {10..4}`
ใน Event-B	บรรทัด 43 `act9: t_time_1 :∈ 5..1` บรรทัด 46 `act12: t_time_3 :∈ 10..4`
PO ที่ล้ม (2)	`INITIALISATION/act9/FIS` — `act9: t_time_1 :∈ 5..1` `INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 10..4`
ทำไม	สูตรแปลงได้ `:∈ 5..1` ซึ่งไม่มีค่าให้เลือก (5 > 1); สูตรแปลงได้ `:∈ 10..4` ซึ่งไม่มีค่าให้เลือก (10 > 4) → FIS ล้ม

SmartStreetLight

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน/fail/SmartStreetLight/

ใน diagram	ช่วงเวลากลับด้าน: `notifyDim(faultValue) {5..1}`, `approveDim() {5..1}`
ใน Event-B	บรรทัด 81 `act28: t_time_9 :∈ 5..1` บรรทัด 84 `act31: t_time_10 :∈ 5..1`
PO ที่ล้ม (2)	`INITIALISATION/act28/FIS` — `act28: t_time_9 :∈ 5..1` `INITIALISATION/act31/FIS` — `act31: t_time_10 :∈ 5..1`
ทำไม	สูตรแปลงได้ `:∈ 5..1` ซึ่งไม่มีค่าให้เลือก (5 > 1); สูตรแปลงได้ `:∈ 5..1` ซึ่งไม่มีค่าให้เลือก (5 > 1) → FIS ล้ม

SolarPanelMonitor

โมเดลผิดจริง · A + B · mod_inverted

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน/fail/SolarPanelMonitor/

ใน diagram	ช่วงเวลากลับด้าน: `reportExport(powerData) {3..0}` alt ใช้เงื่อนไข `[attemptNo mod 2 == 0]` / `[attemptNo mod 2 != 0]` กับตัวนับของ loop `[attemptNo < 8]`
ใน Event-B	บรรทัด 58 `act18: t_time_7 :∈ 3..0` บรรทัด 150 `grd5: attemptNo mod 2 = 0` บรรทัด 168 `grd7: attemptNo mod 2 = 0` … และอีก 4 บรรทัด

PO ที่ล้ม (7)	`INITIALISATION/act18/FIS` — `act18: t_time_7 :∈ 3..0`
	`sendconfirmFault_4/grd5/WD` — `grd5: attemptNo mod 2 = 0`
	`receiveconfirmFault_4/grd7/WD` — `grd7: attemptNo mod 2 = 0`
	… และอีก 4 ข้อ

ทำไม	สูตรแปลงได้ `:∈ 3..0` ซึ่งไม่มีค่าให้เลือก (3 > 0) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `attemptNo mod 2` คำนวณได้เมื่อ `attemptNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `attemptNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

07_ค้าปลีก อาหาร และ e-commerce (5)

CafeLoyaltyPoints

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

07_ค้าปลีก อาหาร และ e-commerce/fail/CafeLoyaltyPoints/

ใน diagram	opt ใช้เงื่อนไข `[roundNo mod 2 == 0]` กับตัวนับของ loop `[roundNo < 12]`
ใน Event-B	บรรทัด 163 `grd5: roundNo mod 2 = 0` บรรทัด 180 `grd7: roundNo mod 2 = 0`
PO ที่ล้ม (2)	`sendissueVisit_4_opt1/grd5/WD` — `grd5: roundNo mod 2 = 0` `receiveissueVisit_4_opt1/grd7/WD` — `grd7: roundNo mod 2 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `roundNo mod 2` คำนวณได้เมื่อ `roundNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `roundNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

GroceryDeliverySlot

โมเดลผิดจริง · A + B · mod_inverted

07_ค้าปลีก อาหาร และ e-commerce/fail/GroceryDeliverySlot/

ใน diagram	ช่วงเวลากลับด้าน: `syncSlot(slotCode) {2..0}` alt ใช้เงื่อนไข `[retryNo mod 3 == 0]` / `[retryNo mod 3 != 0]` กับตัวนับของ loop `[retryNo < 8]`
ใน Event-B	บรรทัด 66 `act16: t_time_6 :∈ 2..0` บรรทัด 174 `grd5: retryNo mod 3 = 0` บรรทัด 191 `grd7: retryNo mod 3 = 0` ... และอีก 14 บรรทัด

PO ที่ล้ม (17)	`INITIALISATION/act16/FIS` — `act16: t_time_6 :∈ 2..0`
	`sendqueryOrder_4/grd5/WD` — `grd5: retryNo mod 3 = 0`
	`receivequeryOrder_4/grd7/WD` — `grd7: retryNo mod 3 = 0`
	... และอีก 14 ข้อ

ทำไม	สูตรแปลงได้ `:∈ 2..0` ซึ่งไม่มีค่าให้เลือก (2 > 0) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `retryNo mod 3` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

OnlineAuction model_030

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

07_ค้าปลีก อาหาร และ e-commerce/fail/OnlineAuction/

ใน diagram	ช่วงเวลากลับด้าน: `raiseBid(bidAmount) {10..8}`
ใน Event-B	บรรทัด 49 `act12: t_time_4 :∈ 10..8`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_4 :∈ 10..8`
ทำไม	สูตรแปลงได้ `:∈ 10..8` ซึ่งไม่มีค่าให้เลือก (10 > 8) → FIS ล้ม

SmartVendingFridge

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

07_ค้าปลีก อาหาร และ e-commerce/fail/SmartVendingFridge/

ใน diagram	opt ใช้เงื่อนไข `[attemptNo mod 4 == 0]` กับตัวนับของ loop `[attemptNo < 9]`
ใน Event-B	บรรทัด 132 `grd5: attemptNo mod 4 = 0` บรรทัด 149 `grd7: attemptNo mod 4 = 0` บรรทัด 161 `grd5: attemptNo mod 4 = 0` บรรทัด 179 `grd7: attemptNo mod 4 = 0`

PO ที่ล้ม (4)	`sendfetchItem_4_opt1/grd5/WD` — `grd5: attemptNo mod 4 = 0`
	`receivefetchItem_4_opt1/grd7/WD` — `grd7: attemptNo mod 4 = 0`
	`sendsendRestock_5_opt1/grd5/WD` — `grd5: attemptNo mod 4 = 0`
	`receivesendRestock_5_opt1/grd7/WD` — `grd7: attemptNo mod 4 = 0`

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `attemptNo mod 4` คำนวณได้เมื่อ `attemptNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `attemptNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

ToolRentalLocker model_044

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

07_ค้าปลีก อาหาร และ e-commerce/fail/ToolRentalLocker/

ใน diagram	ช่วงเวลากลับด้าน: `reserveTool(toolID) {5..3}`
ใน Event-B	บรรทัด 46 `act9: t_time_1 :∈ 5..3`
PO ที่ล้ม (1)	`INITIALISATION/act9/FIS` — `act9: t_time_1 :∈ 5..3`
ทำไม	สูตรแปลงได้ `:∈ 5..3` ซึ่งไม่มีค่าให้เลือก (5 > 3) → FIS ล้ม

08_การจอง ที่พัก และกิจกรรม (6)

ConcertTicketQueue

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

08_การจอง ที่พัก และกิจกรรม/fail/ConcertTicketQueue/

ใน diagram	ช่วงเวลากลับด้าน: `syncSeat() {10..6}`
ใน Event-B	บรรทัด 44 `act8: t_time_1 :∈ 10..6`
PO ที่ล้ม (1)	`INITIALISATION/act8/FIS` — `act8: t_time_1 :∈ 10..6`
ทำไม	สูตรแปลงได้ `:∈ 10..6` ซึ่งไม่มีค่าให้เลือก (10 > 6) → FIS ล้ม

ConferenceRoomBooking model_012

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

08_การจอง ที่พัก และกิจกรรม/fail/ConferenceRoomBooking/

ใน diagram	ช่วงเวลากลับด้าน: `updatePanel(eventID) {2..1}`
ใน Event-B	บรรทัด 52 `act15: t_time_6 :∈ 2..1`
PO ที่ล้ม (1)	`INITIALISATION/act15/FIS` — `act15: t_time_6 :∈ 2..1`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม

EventCheckInQR

โมเดลผิดจริง · A + B · mod_inverted

08_การจอง ที่พัก และกิจกรรม/fail/EventCheckInQR/

ใน diagram	ช่วงเวลากลับด้าน: `issuePass(passValue) {60..6}` alt ใช้เงื่อนไข `[passNo mod 4 == 0] / [passNo mod 4 != 0]` กับตัวนับของ loop `[passNo < 8]`
ใน Event-B	บรรทัด 66 `act22: t_time_12 :∈ 60..6` บรรทัด 159 `grd5: passNo mod 4 = 0` บรรทัด 177 `grd7: passNo mod 4 = 0` ... และอีก 4 บรรทัด

PO ที่ล้ม (7)	`INITIALISATION/act22/FIS` — `act22: t_time_12 :∈ 60..6`
	`sendsendBadge_4/grd5/WD` — `grd5: passNo mod 4 = 0`
	`receivesendBadge_4/grd7/WD` — `grd7: passNo mod 4 = 0`
	... และอีก 4 ข้อ

ทำไม	สูตรแปลงได้ `:∈ 60..6` ซึ่งไม่มีค่าให้เลือก (60 > 6) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `passNo mod 4` คำนวณได้เมื่อ `passNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `passNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

GolfCartFleet

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

08_การจอง ที่พัก และกิจกรรม/fail/GolfCartFleet/

ใน diagram	ช่วงเวลากลับด้าน: `confirmCart(feeData) {8..6}`
ใน Event-B	บรรทัด 38 `act8: t_time_4 :∈ 8..6`
PO ที่ล้ม (1)	`INITIALISATION/act8/FIS` — `act8: t_time_4 :∈ 8..6`
ทำไม	สูตรแปลงได้ `:∈ 8..6` ซึ่งไม่มีค่าให้เลือก (8 > 6) → FIS ล้ม

GymCheckIn model_022

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

08_การจอง ที่พัก และกิจกรรม/fail/GymCheckIn/

ใน diagram	ช่วงเวลากลับด้าน: `scanQR(memberCode) {2..0}`
ใน Event-B	บรรทัด 49 `act9: t_time_1 :∈ 2..0`
PO ที่ล้ม (1)	`INITIALISATION/act9/FIS` — `act9: t_time_1 :∈ 2..0`
ทำไม	สูตรแปลงได้ `:∈ 2..0` ซึ่งไม่มีค่าให้เลือก (2 > 0) → FIS ล้ม

MarathonTiming

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

08_การจอง ที่พัก และกิจกรรม/fail/MarathonTiming/

ใน diagram	opt ใช้เงื่อนไข `[roundNo mod 2 == 0]` กับตัวนับของ loop `[roundNo < 5]`
ใน Event-B	บรรทัด 155 `grd5: roundNo mod 2 = 0` บรรทัด 173 `grd7: roundNo mod 2 = 0`
PO ที่ล้ม (2)	`sendrequestBib_4_opt1/grd5/WD` — `grd5: roundNo mod 2 = 0` `receiverequestBib_4_opt1/grd7/WD` — `grd7: roundNo mod 2 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `roundNo mod 2` คำนวณได้เมื่อ `roundNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `roundNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

09_การศึกษาและบริการภาครัฐ (7)

BoardingSchoolGate

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

09_การศึกษาและบริการภาครัฐ/fail/BoardingSchoolGate/

ใน diagram	ช่วงเวลากลับด้าน: `requestExit(noticeRef) {30..20}`
ใน Event-B	บรรทัด 66 `act16: t_time_3 :∈ 30..20`
PO ที่ล้ม (1)	`INITIALISATION/act16/FIS` — `act16: t_time_3 :∈ 30..20`
ทำไม	สูตรแปลงได้ `:∈ 30..20` ซึ่งไม่มีค่าให้เลือก (30 > 20) → FIS ล้ม

ChildcareCheckin

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

09_การศึกษาและบริการภาครัฐ/fail/ChildcareCheckin/

ใน diagram	ช่วงเวลากลับด้าน: `cancelChild(pickupValue) {5..4}`
ใน Event-B	บรรทัด 54 `act16: t_time_9 :∈ 5..4`
PO ที่ล้ม (1)	`INITIALISATION/act16/FIS` — `act16: t_time_9 :∈ 5..4`
ทำไม	สูตรแปลงได้ `:∈ 5..4` ซึ่งไม่มีค่าให้เลือก (5 > 4) → FIS ล้ม

CourseEnrollment model_013

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

09_การศึกษาและบริการภาครัฐ/fail/CourseEnrollment/

ใน diagram	ช่วงเวลากลับด้าน: `checkPrereq(courseID) {5..2}`, `requestApproval(courseID) {48..18}`
ใน Event-B	บรรทัด 49 `act12: t_time_4 :∈ 5..2` บรรทัด 52 `act15: t_time_6 :∈ 48..18`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_4 :∈ 5..2` `INITIALISATION/act15/FIS` — `act15: t_time_6 :∈ 48..18`
ทำไม	สูตรแปลงได้ `:∈ 5..2` ซึ่งไม่มีค่าให้เลือก (5 > 2); สูตรแปลงได้ `:∈ 48..18` ซึ่งไม่มีค่าให้เลือก (48 > 18) → FIS ล้ม

DigitalLibraryAccess

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

09_การศึกษาและบริการภาครัฐ/fail/DigitalLibraryAccess/

ใน diagram	ช่วงเวลากลับด้าน: `sendLicense(loanValue) {20..11}`, `releaseLoan(ebookInfo) {30..12}`, `checkEbook(ebookRef) {60..9}`
ใน Event-B	บรรทัด 47 `act8: t_time_3 :∈ 20..11` บรรทัด 53 `act14: t_time_5 :∈ 30..12` บรรทัด 56 `act17: t_time_6 :∈ 60..9`

PO ที่ล้ม (3)	`INITIALISATION/act8/FIS` — `act8: t_time_3 :∈ 20..11`
	`INITIALISATION/act14/FIS` — `act14: t_time_5 :∈ 30..12`
	`INITIALISATION/act17/FIS` — `act17: t_time_6 :∈ 60..9`

ทำไม	สูตรแปลงได้ `:∈ 20..11` ซึ่งไม่มีค่าให้เลือก (20 > 11); สูตรแปลงได้ `:∈ 30..12` ซึ่งไม่มีค่าให้เลือก (30 > 12); สูตรแปลงได้ `:∈ 60..9` ซึ่งไม่มีค่าให้เลือก (60 > 9) → FIS ล้ม

PrisonVisitBooking

โมเดลผิดจริง · A + B · mod_inverted

09_การศึกษาและบริการภาครัฐ/fail/PrisonVisitBooking/

ใน diagram	ช่วงเวลากลับด้าน: `requestClearance() {2..1}` opt ใช้เงื่อนไข `[passNo mod 2 == 0]` กับตัวนับของ loop `[passNo < 9]`
ใน Event-B	บรรทัด 63 `act19: t_time_7 :∈ 2..1` บรรทัด 159 `grd5: passNo mod 2 = 0` บรรทัด 176 `grd7: passNo mod 2 = 0`

PO ที่ล้ม (3)	`INITIALISATION/act19/FIS` — `act19: t_time_7 :∈ 2..1`
	`sendcancelSlot_4_opt1/grd5/WD` — `grd5: passNo mod 2 = 0`
	`receivecancelSlot_4_opt1/grd7/WD` — `grd7: passNo mod 2 = 0`

ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `passNo mod 2` คำนวณได้เมื่อ `passNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `passNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

RemoteExamProctor

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

09_การศึกษาและบริการภาครัฐ/fail/RemoteExamProctor/

ใน diagram	ช่วงเวลากลับด้าน: `scheduleSession(examData, sessionID) {30..18}`, `notifyFlag(sessionInfo) {15..6}`
ใน Event-B	บรรทัด 49 `act12: t_time_3 :∈ 30..18` บรรทัด 52 `act15: t_time_6 :∈ 15..6`
PO ที่ล้ม (2)	`INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 30..18` `INITIALISATION/act15/FIS` — `act15: t_time_6 :∈ 15..6`
ทำไม	สูตรแปลงได้ `:∈ 30..18` ซึ่งไม่มีค่าให้เลือก (30 > 18); สูตรแปลงได้ `:∈ 15..6` ซึ่งไม่มีค่าให้เลือก (15 > 6) → FIS ล้ม

SmartLibraryShelf

โมเดลผิดจริง · A + B · mod_inverted

09_การศึกษาและบริการภาครัฐ/fail/SmartLibraryShelf/

ใน diagram	ช่วงเวลากลับด้าน: `requestHold(bookCode, holdInfo) {10..0}` opt ใช้เงื่อนไข `[roundNo mod 4 == 0]` กับตัวนับของ loop `[roundNo < 12]`
ใน Event-B	บรรทัด 66 `act19: t_time_9 :∈ 10..0` บรรทัด 160 `grd5: roundNo mod 4 = 0` บรรทัด 178 `grd7: roundNo mod 4 = 0` ... และอีก 2 บรรทัด

PO ที่ล้ม (5)	`INITIALISATION/act19/FIS` — `act19: t_time_9 :∈ 10..0`
	`sendrequestBook_4_opt1/grd5/WD` — `grd5: roundNo mod 4 = 0`
	`receiverequestBook_4_opt1/grd7/WD` — `grd7: roundNo mod 4 = 0`
	... และอีก 2 ข้อ

ทำไม	สูตรแปลงได้ `:∈ 10..0` ซึ่งไม่มีค่าให้เลือก (10 > 0) → FIS ล้ม สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `roundNo mod 4` คำนวณได้เมื่อ `roundNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `roundNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

10_ไอทีและบริการดิจิทัล (9)

BulkSMSGateway

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

10_ไอทีและบริการดิจิทัล/fail/BulkSMSGateway/

ใน diagram	opt ใช้เงื่อนไข `[retryNo mod 3 == 0]` กับตัวนับของ loop `[retryNo < 11]`
ใน Event-B	บรรทัด 138 `grd5: retryNo mod 3 = 0` บรรทัด 155 `grd7: retryNo mod 3 = 0`
PO ที่ล้ม (2)	`sendlogDelivery_4_opt1/grd5/WD` — `grd5: retryNo mod 3 = 0` `receivelogDelivery_4_opt1/grd7/WD` — `grd7: retryNo mod 3 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `retryNo mod 3` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

ChatMessaging model_008

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted3

10_ไอทีและบริการดิจิทัล/fail/ChatMessaging/

ใน diagram	ช่วงเวลากลับด้าน: `deliverText(textMsg) {5..1}`, `replyText(replyMsg) {10..5}`, `deliverReply(replyMsg) {5..3}`
ใน Event-B	บรรทัด 52 `act12: t_time_2 :∈ 5..1` บรรทัด 55 `act15: t_time_7 :∈ 10..5` บรรทัด 58 `act18: t_time_8 :∈ 5..3`

PO ที่ล้ม (3)	`INITIALISATION/act12/FIS` — `act12: t_time_2 :∈ 5..1`
	`INITIALISATION/act15/FIS` — `act15: t_time_7 :∈ 10..5`
	`INITIALISATION/act18/FIS` — `act18: t_time_8 :∈ 5..3`

ทำไม	สูตรแปลงได้ `:∈ 5..1` ซึ่งไม่มีค่าให้เลือก (5 > 1); สูตรแปลงได้ `:∈ 10..5` ซึ่งไม่มีค่าให้เลือก (10 > 5); สูตรแปลงได้ `:∈ 5..3` ซึ่งไม่มีค่าให้เลือก (5 > 3) → FIS ล้ม

LoginRateLimiter model_028

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

10_ไอทีและบริการดิจิทัล/fail/LoginRateLimiter/

ใน diagram	opt ใช้เงื่อนไข `[failCount mod 2 == 0]` กับตัวนับของ loop `[failCount < 5]`
ใน Event-B	บรรทัด 199 `grd4: failCount mod 2 = 0` บรรทัด 216 `grd6: failCount mod 2 = 0`
PO ที่ล้ม (2)	`sendlockAccount_6_opt1/grd4/WD` — `grd4: failCount mod 2 = 0` `receivelockAccount_6_opt1/grd6/WD` — `grd6: failCount mod 2 = 0`
ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `failCount mod 2` คำนวณได้เมื่อ `failCount ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `failCount ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

PodcastPublishing

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

10_ไอทีและบริการดิจิทัล/fail/PodcastPublishing/

ใน diagram	alt ใช้เงื่อนไข `[retryNo mod 3 == 0]` / `[retryNo mod 3 != 0]` กับตัวนับของ loop `[retryNo < 3]`
ใน Event-B	บรรทัด 144 `grd5: retryNo mod 3 = 0` บรรทัด 161 `grd7: retryNo mod 3 = 0` บรรทัด 174 `grd5: retryNo mod 3 = 0` ... และอีก 3 บรรทัด

PO ที่ล้ม (6)	`sendreserveEpisode_4/grd5/WD` — `grd5: retryNo mod 3 = 0`
	`receivereserveEpisode_4/grd7/WD` — `grd7: retryNo mod 3 = 0`
	`sendissueAudio_5/grd5/WD` — `grd5: retryNo mod 3 = 0`
	... และอีก 3 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `retryNo mod 3` คำนวณได้เมื่อ `retryNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `retryNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

PrinterQueue model_034

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

10_ไอทีและบริการดิจิทัล/fail/PrinterQueue/

ใน diagram	ช่วงเวลากลับด้าน: `pagePrinted(pageNo) {3..2}`
ใน Event-B	บรรทัด 48 `act13: t_time_3 :∈ 3..2`
PO ที่ล้ม (1)	`INITIALISATION/act13/FIS` — `act13: t_time_3 :∈ 3..2`
ทำไม	สูตรแปลงได้ `:∈ 3..2` ซึ่งไม่มีค่าให้เลือก (3 > 2) → FIS ล้ม

PrintFleetMonitor

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

10_ไอทีและบริการดิจิทัล/fail/PrintFleetMonitor/

ใน diagram	ช่วงเวลากลับด้าน: `fetchToner(tonerInfo) {5..3}`
ใน Event-B	บรรทัด 55 `act15: t_time_5 :∈ 5..3`
PO ที่ล้ม (1)	`INITIALISATION/act15/FIS` — `act15: t_time_5 :∈ 5..3`
ทำไม	สูตรแปลงได้ `:∈ 5..3` ซึ่งไม่มีค่าให้เลือก (5 > 3) → FIS ล้ม

PublicWifiHotspot

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

10_ไอทีและบริการดิจิทัล/fail/PublicWifiHotspot/

ใน diagram	ช่วงเวลากลับด้าน: `issueSession() {15..6}`
ใน Event-B	บรรทัด 59 `act17: t_time_11 :∈ 15..6`
PO ที่ล้ม (1)	`INITIALISATION/act17/FIS` — `act17: t_time_11 :∈ 15..6`
ทำไม	สูตรแปลงได้ `:∈ 15..6` ซึ่งไม่มีค่าให้เลือก (15 > 6) → FIS ล้ม

SatelliteGroundStation

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted1

10_ไอทีและบริการดิจิทัล/fail/SatelliteGroundStation/

ใน diagram	ช่วงเวลากลับด้าน: `assignTelemetry() {30..21}`
ใน Event-B	บรรทัด 66 `act19: t_time_4 :∈ 30..21`
PO ที่ล้ม (1)	`INITIALISATION/act19/FIS` — `act19: t_time_4 :∈ 30..21`
ทำไม	สูตรแปลงได้ `:∈ 30..21` ซึ่งไม่มีค่าให้เลือก (30 > 21) → FIS ล้ม

VideoStreaming

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_alt

10_ไอทีและบริการดิจิทัล/fail/VideoStreaming/

ใน diagram	alt ใช้เงื่อนไข `[stepNo mod 2 == 0] / [stepNo mod 2 != 0]` กับตัวนับของ loop `[stepNo < 7]`
ใน Event-B	บรรทัด 159 `grd5: stepNo mod 2 = 0` บรรทัด 177 `grd7: stepNo mod 2 = 0` บรรทัด 190 `grd5: stepNo mod 2 ≠ 0` ... และอีก 3 บรรทัด

PO ที่ล้ม (6)	`sendvalidateStream_4/grd5/WD` — `grd5: stepNo mod 2 = 0`
	`receivevalidateStream_4/grd7/WD` — `grd7: stepNo mod 2 = 0`
	`sendsendChunk_5/grd5/WD` — `grd5: stepNo mod 2 ≠ 0`
	... และอีก 3 ข้อ

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน alt แต่ `stepNo mod 2` คำนวณได้เมื่อ `stepNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `stepNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

11_เกษตรและสิ่งแวดล้อม (6)

CenterPivotIrrigation

ค่าจริงถูก แต่พิสูจน์ไม่ได้ · B. mod กับตัวนับ loop · mod_opt

11_เกษตรและสิ่งแวดล้อม/fail/CenterPivotIrrigation/

ใน diagram	opt ใช้เงื่อนไข `[cycleNo mod 3 == 0]` กับตัวนับของ loop `[cycleNo < 9]`
ใน Event-B	บรรทัด 150 `grd5: cycleNo mod 3 = 0` บรรทัด 168 `grd7: cycleNo mod 3 = 0` บรรทัด 181 `grd5: cycleNo mod 3 = 0` บรรทัด 199 `grd7: cycleNo mod 3 = 0`

PO ที่ล้ม (4)	`sendreleaseSector_4_opt1/grd5/WD` — `grd5: cycleNo mod 3 = 0`
	`receivereleaseSector_4_opt1/grd7/WD` — `grd7: cycleNo mod 3 = 0`
	`sendupdateWater_5_opt1/grd5/WD` — `grd5: cycleNo mod 3 = 0`
	`receiveupdateWater_5_opt1/grd7/WD` — `grd7: cycleNo mod 3 = 0`

ทำไม	สูตรแปลงใส่เงื่อนไขนี้เป็น guard ของทุก event ใน opt แต่ `cycleNo mod 3` คำนวณได้เมื่อ `cycleNo ≥ 0` เท่านั้น (WD) สูตรแปลงสร้างแค่ `cycleNo ∈ ℤ` — ค่าจริงเริ่มที่ 0 และเพิ่มทีละ 1 จึงไม่ติดลบ แต่ไม่มี invariant บอก prover → WD ล้มทุก guard ที่มี mod

CropDroneSpraying

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

11_เกษตรและสิ่งแวดล้อม/fail/CropDroneSpraying/

ใน diagram	ช่วงเวลากลับด้าน: `syncSpray(areaData, sprayData) {2..1}`
ใน Event-B	บรรทัด 46 `act12: t_time_3 :∈ 2..1`
PO ที่ล้ม (1)	`INITIALISATION/act12/FIS` — `act12: t_time_3 :∈ 2..1`
ทำไม	สูตรแปลงได้ `:∈ 2..1` ซึ่งไม่มีค่าให้เลือก (2 > 1) → FIS ล้ม

ForestFireSensor

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

11_เกษตรและสิ่งแวดล้อม/fail/ForestFireSensor/

ใน diagram	ช่วงเวลากลับด้าน: `notifyAlert(smokeRef) {30..6}`, `reserveAlert(smokeRef) {5..3}`
ใน Event-B	บรรทัด 65 `act17: t_time_6 :∈ 30..6` บรรทัด 68 `act20: t_time_7 :∈ 5..3`
PO ที่ล้ม (2)	`INITIALISATION/act17/FIS` — `act17: t_time_6 :∈ 30..6` `INITIALISATION/act20/FIS` — `act20: t_time_7 :∈ 5..3`
ทำไม	สูตรแปลงได้ `:∈ 30..6` ซึ่งไม่มีค่าให้เลือก (30 > 6); สูตรแปลงได้ `:∈ 5..3` ซึ่งไม่มีค่าให้เลือก (5 > 3) → FIS ล้ม

LivestockFeeder

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

11_เกษตรและสิ่งแวดล้อม/fail/LivestockFeeder/

ใน diagram	ช่วงเวลากลับด้าน: `updateRation(orderInfo, rationRef) {3..1}`
ใน Event-B	บรรทัด 57 `act16: t_time_4 :∈ 3..1`
PO ที่ล้ม (1)	`INITIALISATION/act16/FIS` — `act16: t_time_4 :∈ 3..1`
ทำไม	สูตรแปลงได้ `:∈ 3..1` ซึ่งไม่มีค่าให้เลือก (3 > 1) → FIS ล้ม

RecyclingKiosk

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted2

11_เกษตรและสิ่งแวดล้อม/fail/RecyclingKiosk/

ใน diagram	ช่วงเวลากลับด้าน: `scheduleVoucher() {60..31}`, `queryCredit(voucherInfo) {5..4}`
ใน Event-B	บรรทัด 59 `act17: t_time_9 :∈ 60..31` บรรทัด 62 `act20: t_time_10 :∈ 5..4`
PO ที่ล้ม (2)	`INITIALISATION/act17/FIS` — `act17: t_time_9 :∈ 60..31` `INITIALISATION/act20/FIS` — `act20: t_time_10 :∈ 5..4`
ทำไม	สูตรแปลงได้ `:∈ 60..31` ซึ่งไม่มีค่าให้เลือก (60 > 31); สูตรแปลงได้ `:∈ 5..4` ซึ่งไม่มีค่าให้เลือก (5 > 4) → FIS ล้ม

SmartBeehive

โมเดลผิดจริง · A. ช่วงเวลากลับด้าน · inverted_frame

11_เกษตรและสิ่งแวดล้อม/fail/SmartBeehive/

ใน diagram	ช่วงเวลากลับด้าน: `logWeight() {60..50}`
ใน Event-B	บรรทัด 43 `act9: t_time_3 :∈ 60..50`
PO ที่ล้ม (1)	`INITIALISATION/act9/FIS` — `act9: t_time_3 :∈ 60..50`
ทำไม	สูตรแปลงได้ `:∈ 60..50` ซึ่งไม่มีค่าให้เลือก (60 > 50) → FIS ล้ม

8. โมเดล pass

diagram ถูก ไม่มีช่วงเวลากลับด้าน และไม่มี mod — Event-B คือผลของสูตรแปลงตรง ๆ

01_การเงินและการชำระเงิน (9)

ATMWithdrawal
BankBranchQueue
CanteenPayment
GasStationPrepay
InsuranceClaim
InsuranceTelematics
MarketplaceEscrow
OnlineDonation
TradingOrderGateway

02_สุขภาพและการแพทย์ (12)

AmbulanceRouting
BloodBankRequest
FitnessTracker
GlucoseMeterSync
HomeOxygenConcentrator
HospitalBedAllocation
HospitalPharmacyRobot
HospitalTriage
MedicalTranscription
SmartPillBox
ThermalCameraScreening
VeterinaryClinic

03_การเดินทางและยานพาหนะ (18)

AirportShuttle
BikeSharing
CampusShuttleBus
CarRentalBooking
CarRentalReturn
DroneTrafficControl
FerryTicketGate
FlightCheckIn
GarageGateControl
ParkingPermitPortal
RailFareGate
RentalScooter
SchoolBusTracking
ShipLockOperation
SmartParking
StreetParkingMeter
TollPlazaETC
TrafficSignalPriority

04_โลจิสติกส์และคลังสินค้า (9)

CustomsClearance
DroneDelivery
MailroomSorting
PackageTracking
PortCraneControl
SeaportPilotBooking
WarehouseForklift
WarehousePicking
WarehouseRobotFleet

05_บ้านและอาคารอัจฉริยะ (11)

ApartmentIntercom
BuildingVisitorPass
DormLaundry
ElevatorControl
FireSprinklerSystem
IoTDoorbell
PetDoorController
SmartBabyMonitor
SmartDoorLock
ThermostatControl
VoiceAssistant

06_อุตสาหกรรม พลังงาน และโครงสร้างพื้นฐาน (8)

EVBatterySwap
EVChargingSession
FactoryQualityGate
FactoryRobotArm
HomeEnergyBattery
PipelineLeakSCADA
SmartHelmetMining
WindTurbineMonitor

07_ค้าปลีก อาหาร และ e-commerce (9)

CoffeeOrderKiosk
ConcessionStandOrder
FoodOrderApp
FoodTruckOrdering
OnlineGroceryPicking
PhotoPrintKiosk
SelfCheckoutLane
SmartShoppingCart
VendingMachine

08_การจอง ที่พัก และกิจกรรม (13)

ArtGalleryTicketing
CarWashBooking
CinemaTicketing
ConferenceBadgeScan
CoworkingDeskBooking
HotelCheckIn

HotelKeylessEntry
HotelRoomService
MeetingRoomDisplay
MuseumAudioGuide
RestaurantReservation
SmartGymLocker
StadiumEntryGate

09_การศึกษาและบริการภาครัฐ (10)

CourseAutoGrading
DigitalVotingBooth
DisasterAlertBroadcast
EmergencyDispatch
LabEquipmentBooking
LibraryLoan
OnlineExam
PassportRenewalKiosk
StudentAttendance
TaxFilingPortal

10_ไอทีและบริการดิจิทัล (10)

ArchiveDocumentScan
CampusWifiAccess
CloudBackupJob
CloudCIPipeline
EmailVerification
JobApplicationPortal
OnlineQuizGame
ParentalControlRouter
SensorDataPipeline
ServerAutoScaling

11_เกษตรและสิ่งแวดล้อม (9)

AirQualityMonitor
FarmMilkingRobot
GreenhouseLighting
IrrigationZoneControl
SmartGreenhouse
SmartRainGauge
SmartTrashCompactor
SmartWasteBin
SoilTestingLab